

Documentation technique - DataShare

MVP de transfert de fichiers sécurisé

Élément	Valeur
Projet	DataShare
Type	Application web - MVP
Frontend	React
Backend	.NET 8 Web API
Base de données	PostgreSQL
Qualité	38 tests backend, 92,4 % de couverture, 2 tests Cypress
Performance	k6 - 0 % erreur, p95 158,2 ms

Vue simplifiée

Utilisateur	Frontend React	API .NET	PostgreSQL + fichiers
Navigateur	Pages et formulaires	Règles métier et sécurité	Données + fichiers uploadés

Introduction

DataShare est un MVP d'application de transfert de fichiers sécurisé. L'objectif est de permettre à un utilisateur de créer un compte, d'envoyer un fichier, puis de partager un lien de téléchargement.

Le projet reste volontairement simple : il ne cherche pas à couvrir tous les cas d'une application professionnelle complète, mais il valide les fonctionnalités principales attendues pour une première version.

Fonctionnalités principales

- Création de compte et connexion utilisateur.
- Upload de fichiers depuis l'interface web.
- Historique des fichiers envoyés par l'utilisateur connecté.
- Génération d'un lien de téléchargement avec token.
- Possibilité de protéger un téléchargement par mot de passe.
- Expiration des liens de téléchargement.
- Suppression d'un fichier par son propriétaire.
- Gestion des liens invalides ou non retrouvés.

Architecture générale

L'application est séparée en trois parties : un frontend React, une API backend .NET et une base de données PostgreSQL.

Le frontend communique avec le backend via des appels HTTP. Le backend gère les règles métier, l'authentification, les fichiers et l'accès à la base de données.

PostgreSQL stocke les utilisateurs et les métadonnées des fichiers. Les fichiers eux-mêmes sont stockés côté serveur.

Choix techniques

- React a été choisi pour construire une interface web simple et réactive.
- .NET 8 a été utilisé pour le backend car il permet de créer une API structurée avec des contrôleurs, des services et Entity Framework.
- PostgreSQL a été choisi pour stocker les données de manière fiable.
- Docker Compose permet de lancer facilement la base PostgreSQL pendant le développement.
- Cypress, xUnit, ReportGenerator et k6 ont été utilisés pour valider la qualité du projet.

Modèle de données

Le modèle de données reste volontairement limité pour correspondre au périmètre du MVP.

La table Users stocke les informations des utilisateurs : identifiant, email, mot de passe hashé et date de création.

La table StoredFiles stocke les informations des fichiers : nom original, nom stocké, type, taille, chemin relatif, token de téléchargement, date d'upload, date d'expiration et utilisateur propriétaire.

La relation principale est simple : un utilisateur peut posséder plusieurs fichiers.

API principale

- POST /api/auth/register : création d'un compte utilisateur.
- POST /api/auth/login : connexion et récupération d'un token.
- POST /api/files/upload : upload d'un fichier pour un utilisateur connecté.
- GET /api/files/me : récupération des fichiers de l'utilisateur connecté.
- DELETE /api/files/{id} : suppression d'un fichier appartenant à l'utilisateur.
- GET /api/download/{token} : récupération des informations d'un lien de téléchargement.
- POST /api/download/{token} : téléchargement du fichier, avec vérification du mot de passe si nécessaire.
- GET /api/health : vérification que l'API est disponible.

Installation et lancement

- La base PostgreSQL est lancée avec Docker Compose.
- Les migrations Entity Framework permettent de créer la structure de la base.
- Le backend est lancé avec dotnet run depuis le dossier DataShare.Api.
- Le frontend est lancé avec npm run dev depuis le dossier frontend.
- Le frontend doit contenir un fichier .env avec l'URL de l'API, par exemple VITE_API_URL=http://localhost:5000/api.

Qualité et tests

- Les tests backend ont été exécutés avec dotnet test et la couverture avec XPlat Code Coverage.
- Résultat obtenu : 38 tests réussis, 0 échec et 92,4 % de couverture de lignes.
- Les tests end-to-end Cypress valident deux parcours : le parcours principal utilisateur et le cas d'un lien de téléchargement invalide.
- Les tests Cypress ont donné 2 tests réussis sur 2.

Sécurité

- Les dépendances backend ont été contrôlées avec dotnet list package --vulnerable.
- Les dépendances frontend ont été contrôlées avec npm audit.
- Aucune vulnérabilité connue n'a été détectée au moment de la vérification.
- L'application utilise aussi des mesures de base : mots de passe hashés, token de téléchargement, liens expirables et contrôle de propriétaire sur les fichiers.

Performance

- Un test de performance léger a été réalisé avec k6 sur le parcours d'upload.
- Le scénario crée un utilisateur, récupère un token, envoie un petit fichier et vérifie les métadonnées.
- Résultat obtenu : 0 requête échouée, 25 vérifications réussies sur 25, temps de réponse moyen de 53,04 ms et p95 de 158,2 ms.
- Ce test ne remplace pas un audit complet, mais il montre que le MVP reste stable sur une charge légère.

Maintenance

- La maintenance du projet repose sur des contrôles réguliers : tests, scans de sécurité, migrations et mise à jour de la documentation.
- Les tests doivent être relancés à chaque évolution importante.
- Les scans de sécurité doivent être relancés après l'ajout ou la mise à jour de dépendances.
- Une évolution future utile serait l'ajout d'une purge automatique des fichiers expirés.

Utilisation de l'IA générative

L'IA générative a été utilisée comme aide sur une partie limitée du projet, notamment pour accélérer la production de tests et de documentation.

Les résultats proposés ont été relus, adaptés et vérifiés manuellement avant d'être intégrés.

Les corrections ont été faites lorsque les tests ne compilaient pas ou lorsque le comportement ne correspondait pas au projet.

Conclusion

DataShare répond au besoin principal d'un MVP de transfert de fichiers sécurisé.

Le projet dispose d'une architecture simple, de tests automatisés, d'une vérification de sécurité, d'un test de performance et d'une documentation de maintenance.

Les limites du MVP sont identifiées, ce qui permet de prévoir des améliorations futures sans complexifier inutilement la première version.